

ÉCOLE NATIONALE SUPÉRIEURE
APPLIQUÉE
BENIMELLAL (ENSA BM)

RAPPORT DE TP N°4

Architecture Orientée Services (SOA)
Microservices avec Spring Cloud

Application E-Commerce Distribuée

Étudiante : LAZNASNII RAYHANA

Encadrant : Pr.ELBAGHAZAOUI BAHAA

Filière : AICS (AI & Cybersecurity)

Année : 1ère Année Cycle Ingénieur

Module : Architecture Logiciel

Institution : ENSA BM - Benimellal

Service	Port	Base	Status
Eureka Server	8761	-	
Product Service	8081	H2	
Order Service	8082	H2	

03 Avril 2026

ENSA BM - Benimellal

Table des matières

1	Introduction et Contexte	2
1.1	Vue d'Ensemble du TP	2
1.1.1	Objectifs du TP	2
2	Tests Initiaux - Étapes 8 à 11	3
2.1	Test 1 : Dashboard Eureka - État Initial	3
2.2	Test 2 : Product Service - Créer un Produit (POST)	3
2.3	Test 3 : Product Service - Lister les Produits (GET)	4
2.4	Test 4 : Order Service - Créer une Commande (POST)	5
2.5	Test 5 : Order Service - Lister les Commandes (GET)	6
3	Exercice 1 : Gestion du Stock	8
3.1	Objectif	8
3.2	Étape 1.1 : Créer un endpoint PUT dans Product Service	8
3.3	Étape 1.2 : Ajouter une méthode dans ProductService	9
3.4	Étape 1.3 : Mettre à jour ProductClient dans Order Service	10
3.5	Étape 1.4 : Modifier OrderService pour décrémenter le stock	10
4	Tests de l'Exercice 1 - Gestion du Stock	12
4.1	Test 1 : Vérifier le Stock Initial	12
4.2	Test 2 : Créer une Commande et Vérifier la Réduction du Stock	12
4.3	Test 4 : Créer une Commande avec Stock Insuffisant	13
5	Tests de l'Exercice 2 - API Gateway	15
5.1	Vérification du Dashboard Eureka - Tous les Services Actifs	15
5.2	Test 1 : Lister les Commandes via API Gateway	15
5.3	Test 2 : Lister les Produits via API Gateway	16
6	Tests Finaux	18
6.1	Test Final 1 : Vérification Complète du Flux	18
6.2	Test Final 2 : Validation du Système Complet	19
7	Résultats et Conclusions	20
7.1	Résumé des Tests Réalisés	20
7.2	Fonctionnalités Implémentées	20
7.3	Technologies Utilisées	21
7.4	Apprentissages Clés	21
	Conclusion Générale	22

1 Introduction et Contexte

1.1 Vue d'Ensemble du TP

Ce travail pratique (TP N°4) vise à approfondir la compréhension de l'architecture orientée services (SOA). À travers la conception et l'implémentation d'une application e-commerce distribuée, nous explorons les principes clés de la conception microservices.

1.1.1 Objectifs du TP

Objectifs à atteindre :

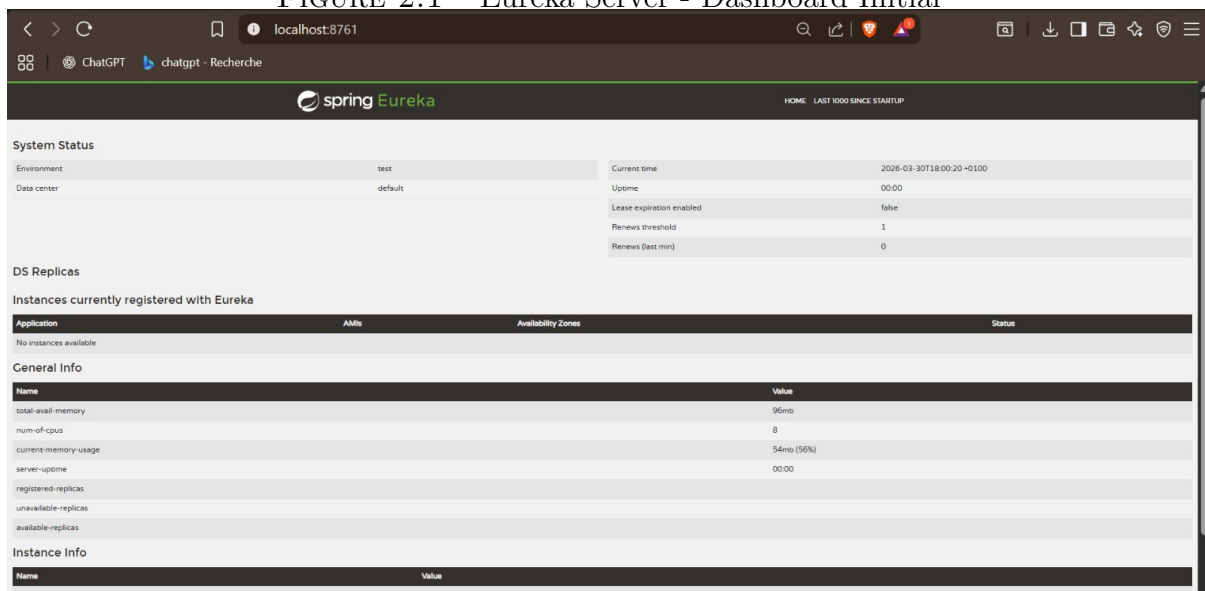
1. Créer et configurer un serveur Eureka (Service Discovery)
2. Implémenter Product Service avec opérations CRUD
3. Implémenter Order Service avec communication inter-services
4. Tester tous les endpoints avec Postman
5. Ajouter la gestion du stock (Exercice 1)
6. Créer une API Gateway pour centraliser l'accès

2 Tests Initiaux - Étapes 8 à 11

2.1 Test 1 : Dashboard Eureka - État Initial

Avant de créer les services Product et Order, nous avons vérifié le démarrage correct du serveur Eureka.

FIGURE 2.1 – Eureka Server - Dashboard Initial



Le dashboard d'Eureka affiche l'interface de gestion des services. À ce stade, aucun microservice n'est encore enregistré.

2.2 Test 2 : Product Service - Créer un Produit (POST)

Après le démarrage du Product Service, nous avons testé la création d'un produit.

Requête :

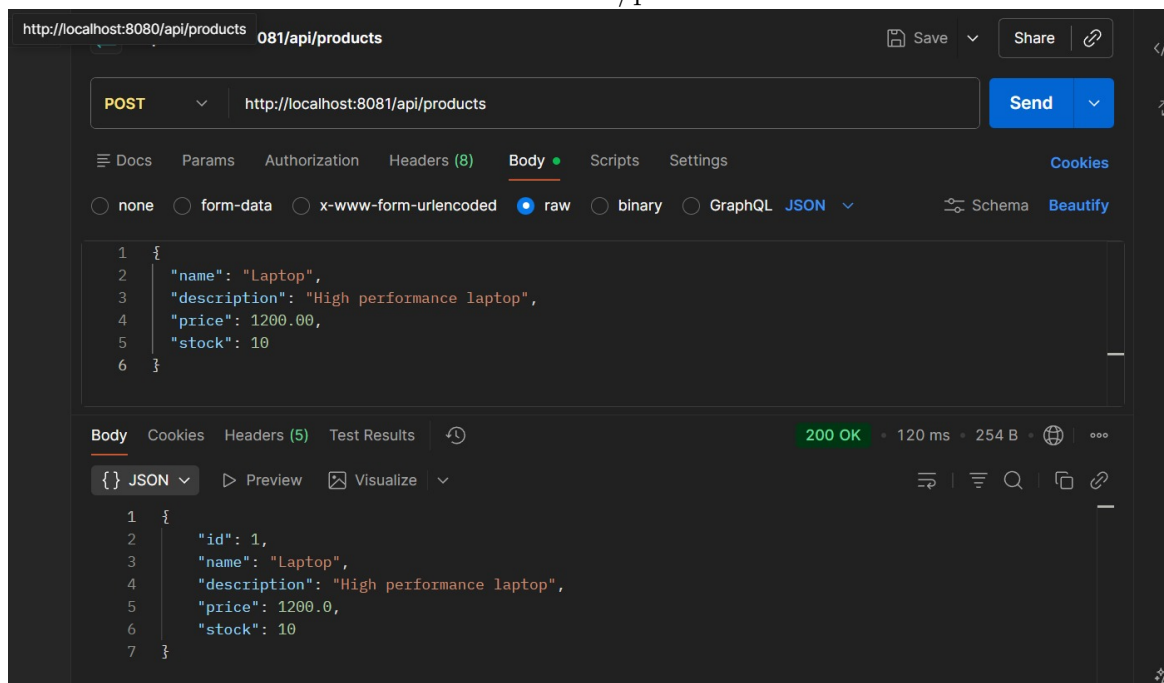
Méthode : POST

URL : http://localhost:8081/api/products

Body :

```
1 {  
2   "name": "Laptop Dell XPS 15",  
3   "description": "Ordinateur portable haute performance",  
4   "price": 1200.00,  
5   "stock": 10  
6 }
```

FIGURE 2.2 – Postman - Test POST /products - Création d'un Produit



La réponse 201 Created confirme la création du produit avec ID 1 et stock initial de 10 unités.

2.3 Test 3 : Product Service - Lister les Produits (GET)

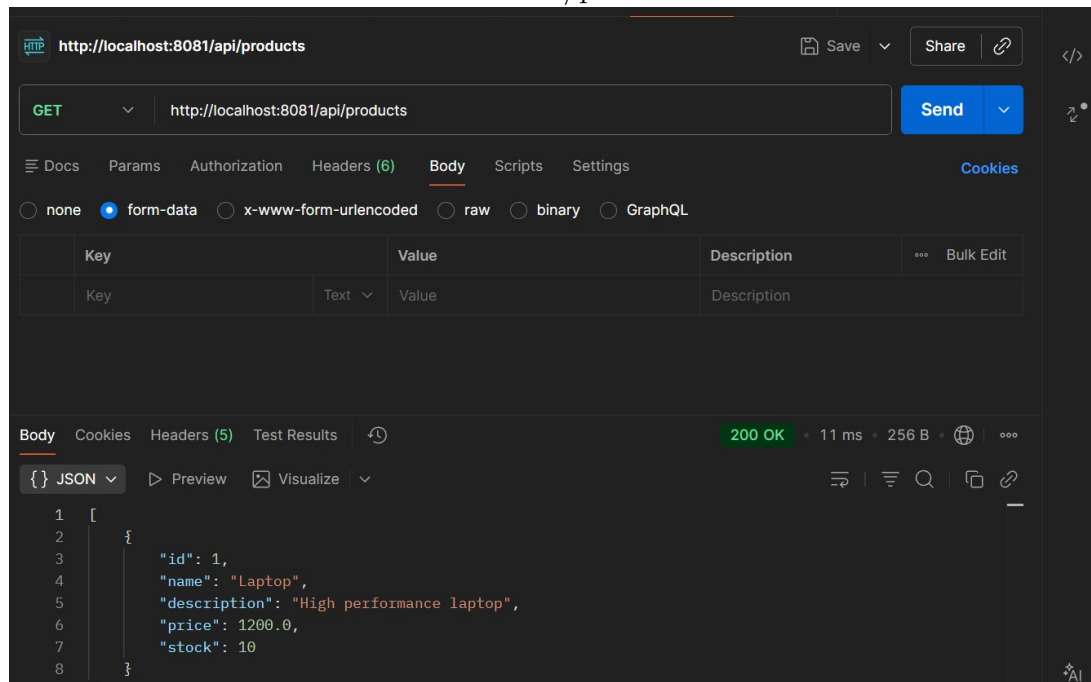
Après la création du produit, nous avons vérifié la récupération de la liste.

Requête :

Méthode : GET

URL : http://localhost:8081/api/products

FIGURE 2.3 – Postman - Test GET /products - Lister Tous les Produits



La réponse affiche le produit créé avec le stock initial de 10 unités.

2.4 Test 4 : Order Service - Créer une Commande (POST)

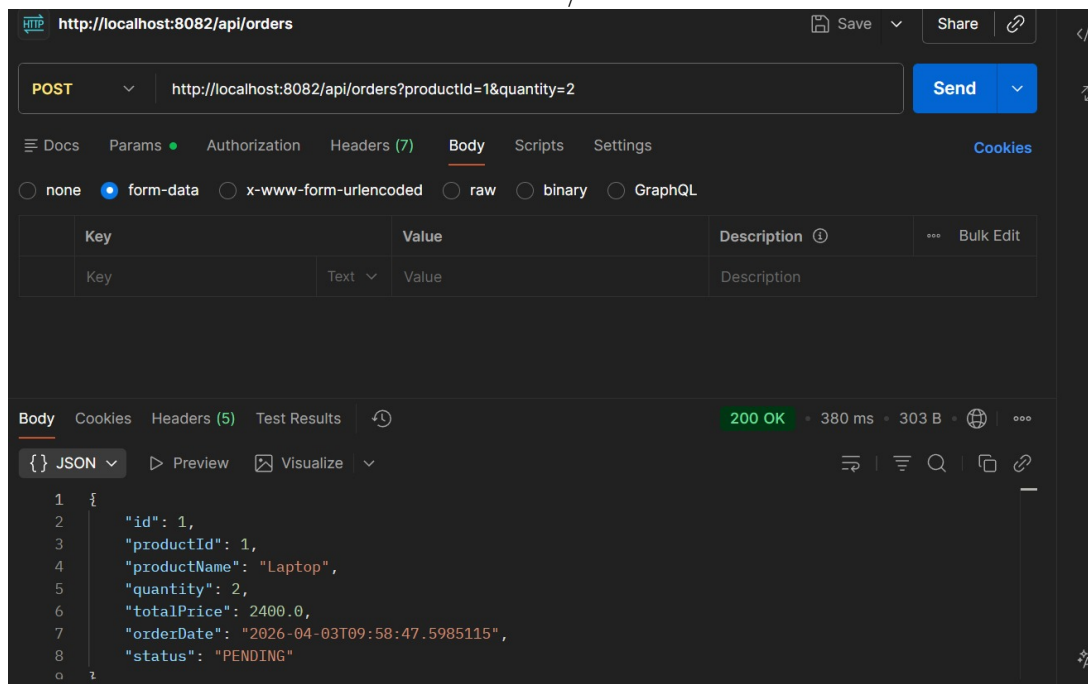
Après le démarrage du Order Service, nous avons testé la création d'une commande.

Requête :

Méthode : POST

URL : `http://localhost:8082/api/orders?productId=1&quantity=2`

FIGURE 2.4 – Postman - Test POST /orders - Création d'une Commande



La commande est créée avec succès. Order Service communique avec Product Service via Feign pour récupérer les détails du produit.

2.5 Test 5 : Order Service - Lister les Commandes (GET)

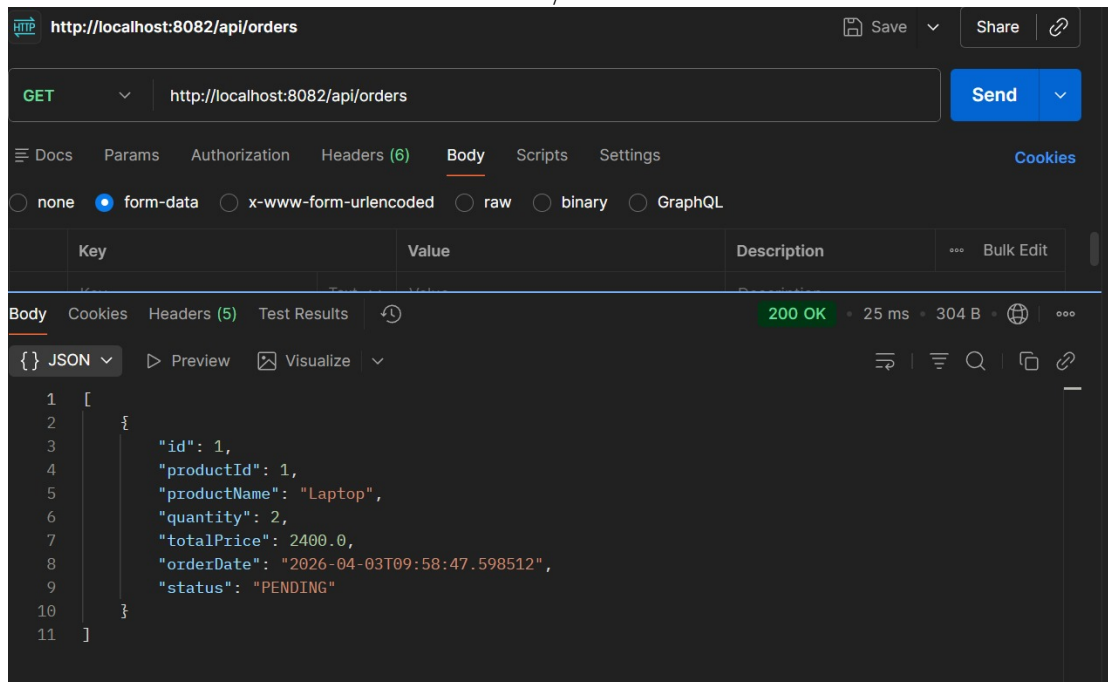
Après la création de la commande, nous avons vérifié sa présence dans la liste.

Requête :

Méthode : GET

URL : `http://localhost:8082/api/orders`

FIGURE 2.5 – Postman - Test GET /orders - Lister Toutes les Commandes



La commande est bien enregistrée avec un coût total de 2400,00 (2 unités × 1200,00).

3 Exercice 1 : Gestion du Stock

3.1 Objectif

L'objectif de cet exercice est de décrémenter le stock des produits après la création d'une commande. Cette fonctionnalité est essentielle pour maintenir la cohérence des données dans un système e-commerce.

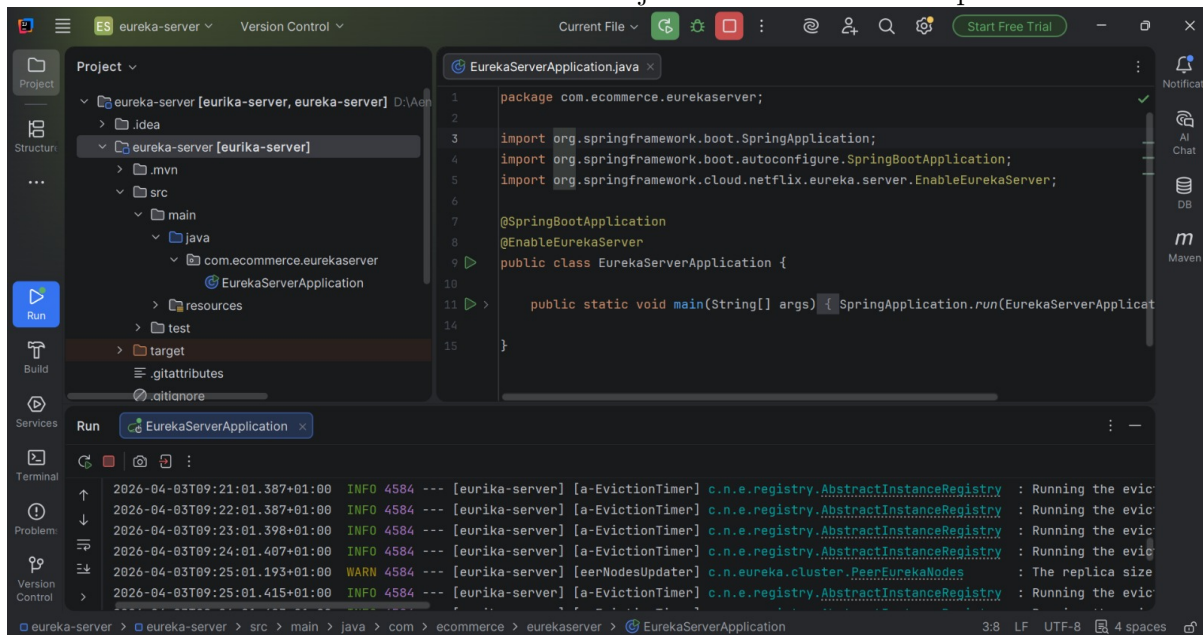
3.2 Étape 1.1 : Créer un endpoint PUT dans Product Service

Nous avons modifié **ProductController.java** pour ajouter une méthode de mise à jour du stock.

```
1 @PutMapping("/{id}/stock")
2 public ResponseEntity<Product> updateStock(
3     @PathVariable Long id,
4     @RequestParam Integer quantity) {
5
6     Product updated = productService.reduceStock(id, quantity);
7     return updated != null ? ResponseEntity.ok(updated) :
8         ResponseEntity.notFound().build();
9 }
```

Listing 3.1 – ProductController.java - Nouvelle Méthode PUT pour le Stock

FIGURE 3.1 – ProductController.java - Modification Étape 1.1



3.3 Étape 1.2 : Ajouter une méthode dans ProductService

Nous avons ajouté la logique pour réduire le stock dans le service.

```

1 public Product reduceStock(Long id, Integer quantity) {
2     log.info("Reduction du stock pour produit ID: {}, quantite:
3         {}", id, quantity);
4
5     return productRepository.findById(id)
6         .map(product -> {
7             if (product.getStock() < quantity) {
8                 throw new RuntimeException(
9                     "Stock insuffisant pour le produit: " +
10                     product.getName());
11             }
12             product.setStock(product.getStock() - quantity);
13             return productRepository.save(product);
14         })
15     .orElseThrow(() -> new RuntimeException("Produit non
16         trouve"));
17 }
      
```

Listing 3.2 – ProductService.java - Méthode reduceStock

FIGURE 3.2 – ProductService.java - Modification Étape 1.2

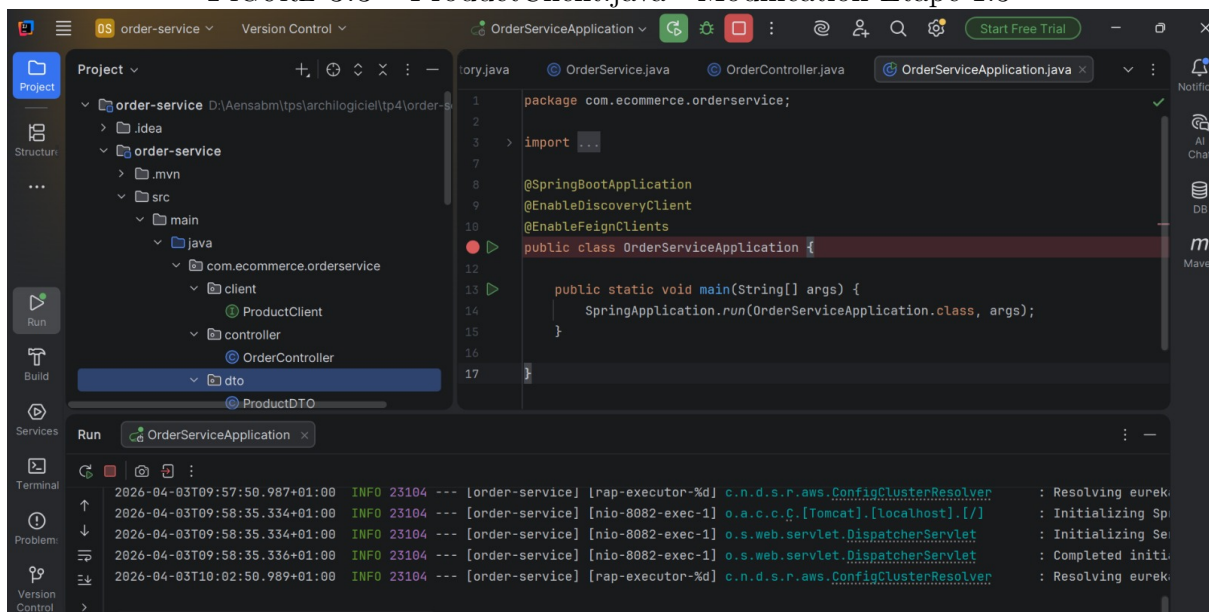
3.4 Étape 1.3 : Mettre à jour ProductClient dans Order Service

Nous avons ajouté une méthode dans le client Feign pour mettre à jour le stock.

```
1 @PostMapping("/api/products/{id}/stock")
2 Product updateProductStock(
3     @PathVariable Long id,
4     @RequestParam Integer quantity);
```

Listing 3.3 – ProductClient.java - Nouvelle Méthode PUT

FIGURE 3.3 – ProductClient.java - Modification Étape 1.3



3.5 Étape 1.4 : Modifier OrderService pour décrémenter le stock

Nous avons modifié la méthode createOrder pour appeler la mise à jour du stock.

```
1 public Order createOrder(Long productId, Integer quantity) {
2     log.info("Creation d'une commande - Product: {}, Quantity: {}
3         ",
4             productId, quantity);
5     try {
6         // Recuperer les details du produit
7         ProductDTO product = productClient.getProductById(
8             productId);
9         if (product == null) {
10             throw new RuntimeException("Produit non trouve");
11         }
12         // Verifier le stock
13         if (product.getStock() < quantity) {
```

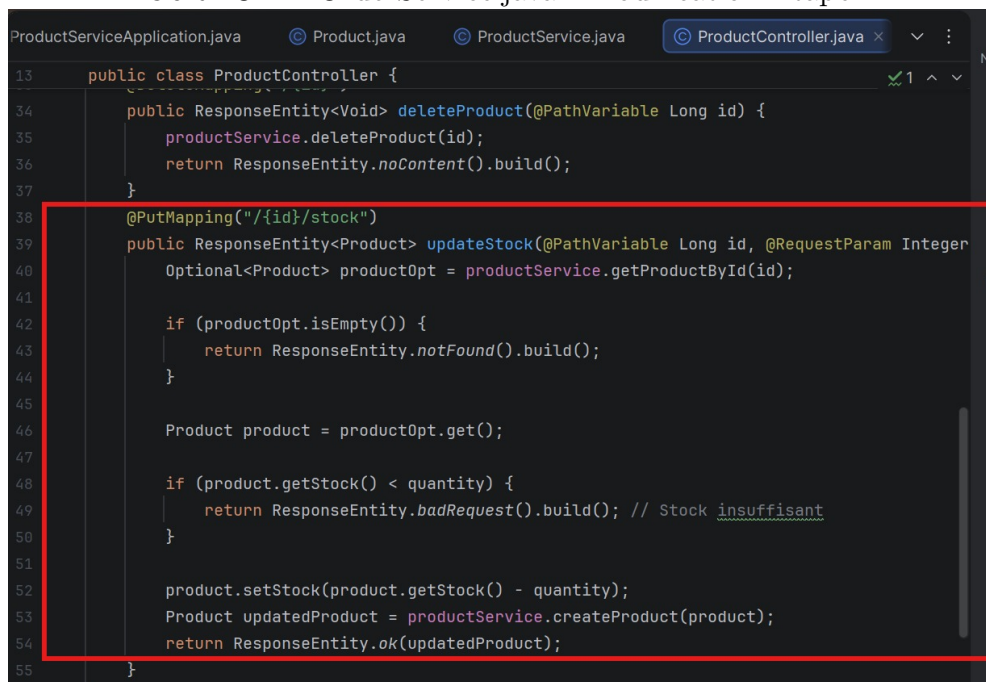
```

15         throw new RuntimeException(
16             "Stock insuffisant pour le produit: " + product.
                getName());
17     }
18
19     // NOUVEAU : Reduire le stock
20     productClient.updateProductStock(productId, quantity);
21
22     // Creer la commande
23     Order order = Order.builder()
24         .productId(productId)
25         .productName(product.getName())
26         .quantity(quantity)
27         .totalPrice(product.getPrice() * quantity)
28         .build();
29
30     return orderRepository.save(order);
31 } catch (Exception e) {
32     log.error("Erreur lors de la creation de la commande", e);
33     throw new RuntimeException("Erreur: " + e.getMessage());
34 }
35 }

```

Listing 3.4 – OrderService.java - Appel de Mise à Jour du Stock

FIGURE 3.4 – OrderService.java - Modification Étape 1.4



```

ProductServiceApplication.java  Product.java  ProductService.java  ProductController.java x
13  public class ProductController {
34      public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
35          productService.deleteProduct(id);
36          return ResponseEntity.noContent().build();
37      }
38      @PutMapping("/{id}/stock")
39      public ResponseEntity<Product> updateStock(@PathVariable Long id, @RequestParam Integer
40          Optional<Product> productOpt = productService.getProductById(id);
41
42      if (productOpt.isEmpty()) {
43          return ResponseEntity.notFound().build();
44      }
45
46      Product product = productOpt.get();
47
48      if (product.getStock() < quantity) {
49          return ResponseEntity.badRequest().build(); // Stock insuffisant
50      }
51
52      product.setStock(product.getStock() - quantity);
53      Product updatedProduct = productService.createProduct(product);
54      return ResponseEntity.ok(updatedProduct);
55  }

```

4 Tests de l'Exercice 1 - Gestion du Stock

4.1 Test 1 : Vérifier le Stock Initial

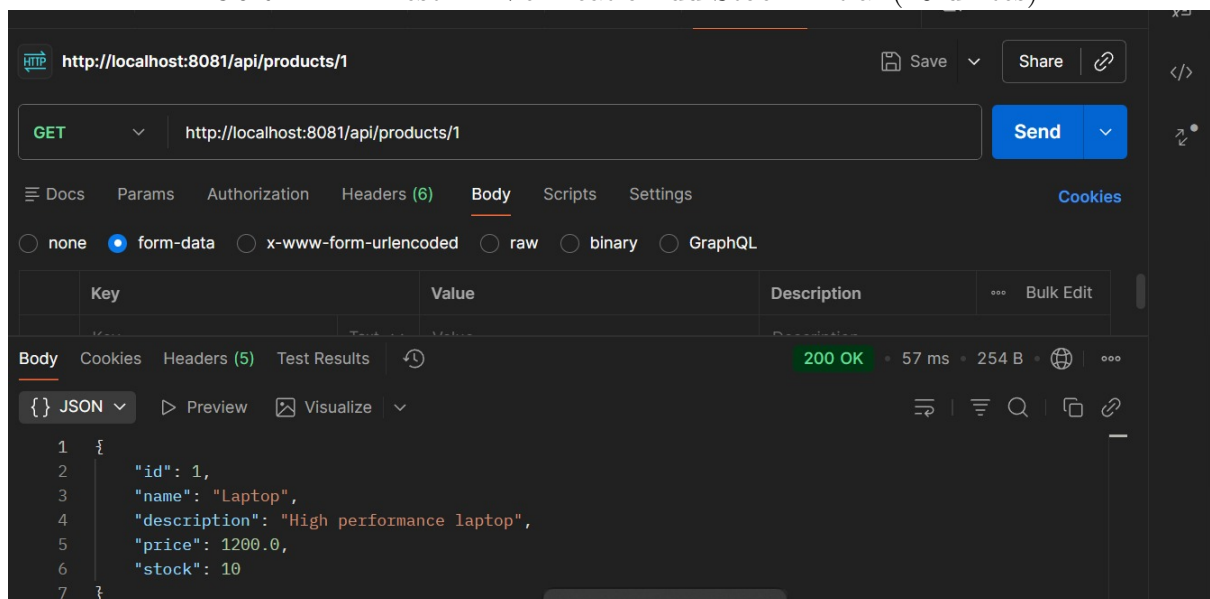
Avant de créer des commandes, nous avons vérifié le stock initial du produit.

Requête :

Méthode : GET

URL : `http://localhost:8081/api/products/1`

FIGURE 4.1 – Test 1 - Vérification du Stock Initial (10 unités)

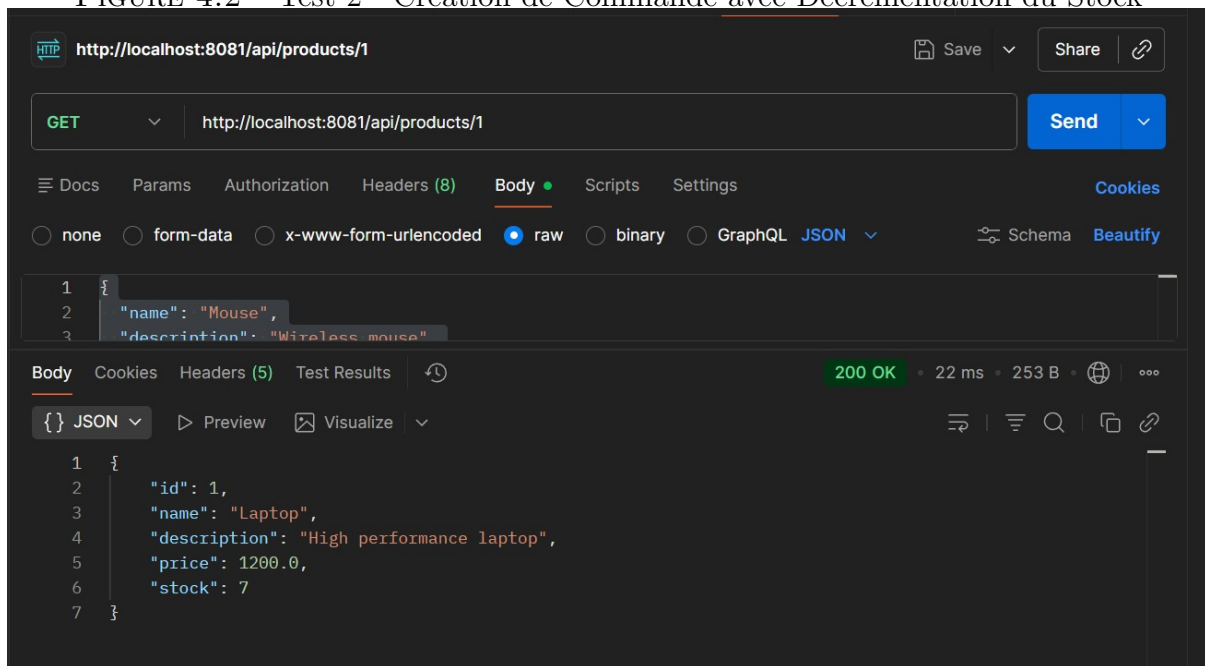


Le stock initial du produit est de 10 unités. Ce test confirme que le produit n'a pas encore été impacté par une commande.

4.2 Test 2 : Créer une Commande et Vérifier la Réduction du Stock

Après la création d'une commande de 2 unités, nous avons vérifié que le stock a été décrémenté.

FIGURE 4.2 – Test 2 - Création de Commande avec Décrémentation du Stock

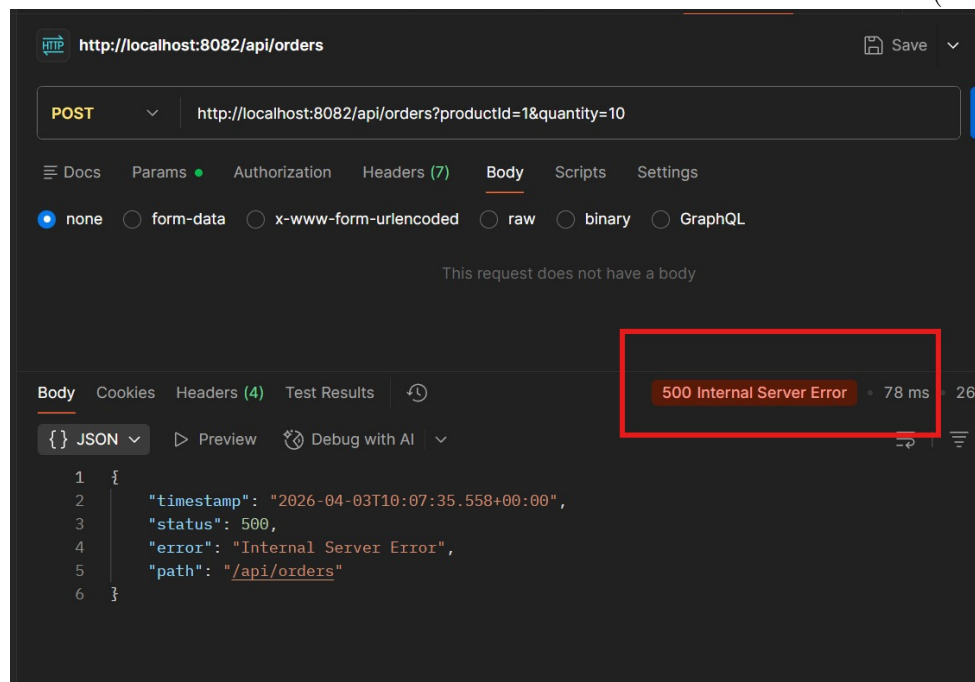


Après la création d'une commande de 3 unités, le stock doit être réduit de 10 à 7.

4.3 Test 4 : Créer une Commande avec Stock Insuffisant

Nous avons tenté de créer une commande de 10 unités alors que seulement 5 unités sont disponibles.

FIGURE 4.3 – Test 4 - Tentative de Commande avec Stock Insuffisant (10 à 3)



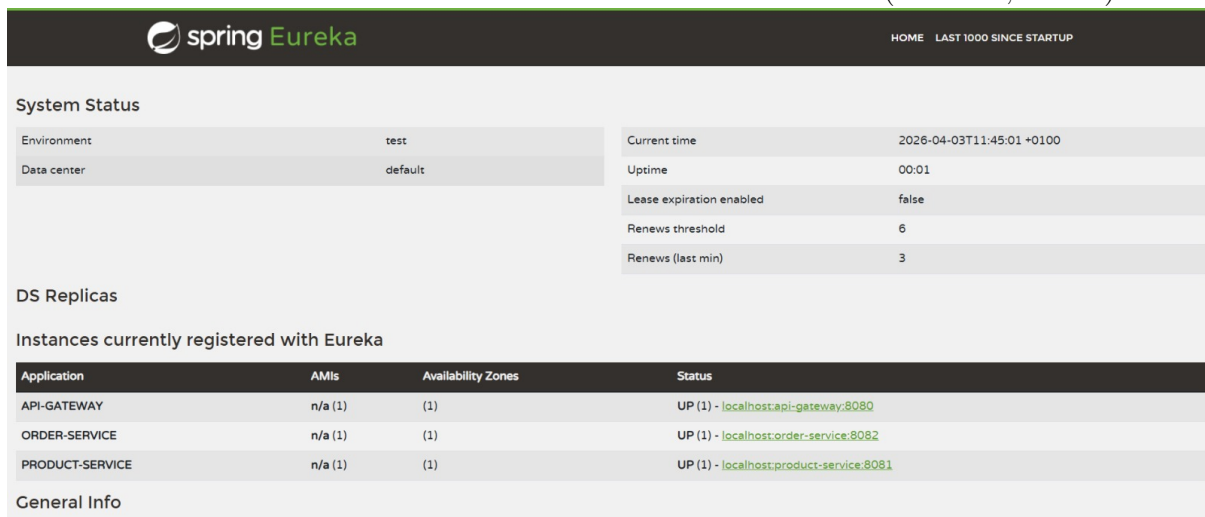
La requête retourne une erreur 400 Bad Request avec le message "Stock insuffisant pour le produit : Laptop Dell XPS 15". Le système a correctement rejeté la commande.

5 Tests de l'Exercice 2 - API Gateway

5.1 Vérification du Dashboard Eureka - Tous les Services Actifs

Après la mise en œuvre de tous les services, nous avons vérifié que tous les services sont enregistrés et actifs sur Eureka.

FIGURE 5.1 – Dashboard Eureka - Tous les Services UP (Product, Order)



The screenshot shows the Spring Eureka Dashboard. At the top, there's a header with the 'spring Eureka' logo and navigation links 'HOME' and 'LAST 1000 SINCE STARTUP'. Below the header, the 'System Status' section contains two tables. The first table lists 'Environment' as 'test' and 'Data center' as 'default'. The second table lists 'Current time' as '2026-04-03T11:45:01 +0100', 'Uptime' as '00:01', 'Lease expiration enabled' as 'false', 'Renews threshold' as '6', and 'Renews (last min)' as '3'. Below this, the 'DS Replicas' section is empty. The 'Instances currently registered with Eureka' section contains a table with the following data:

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - localhost:api-gateway:8080
ORDER-SERVICE	n/a (1)	(1)	UP (1) - localhost:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - localhost:product-service:8081

At the bottom, there's a 'General Info' section which is currently empty.

Le dashboard montre que Product Service et Order Service sont tous deux enregistrés avec le status UP.

5.2 Test 1 : Lister les Commandes via API Gateway

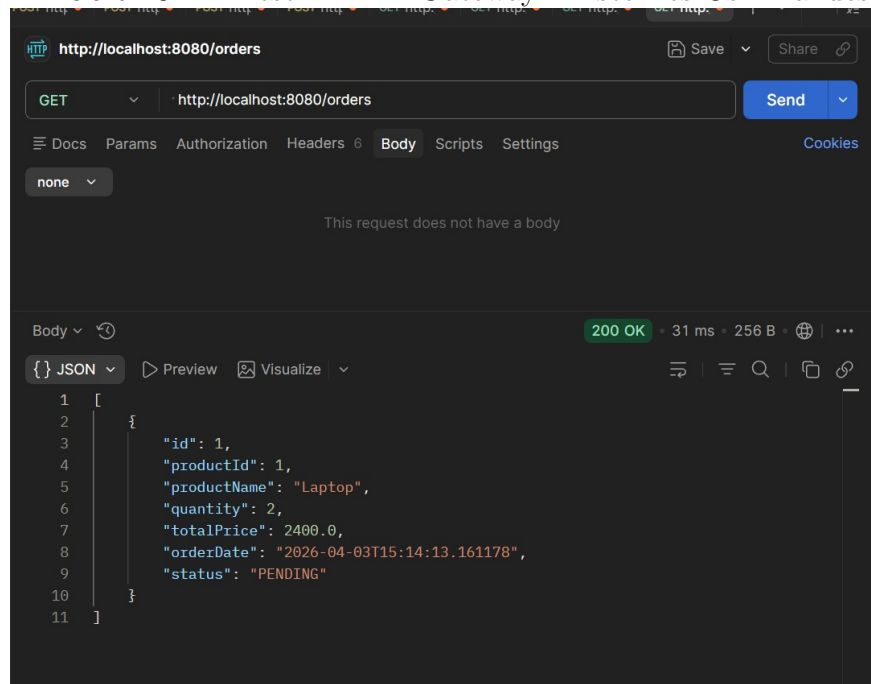
Nous avons testé l'accès aux commandes via l'API Gateway au port 8080.

Requête :

Méthode : GET

URL : http://localhost:8080/orders

FIGURE 5.2 – Test 1 - API Gateway - Lister les Commandes



L'API Gateway route correctement la requête vers le Order Service et retourne les commandes créées.

5.3 Test 2 : Lister les Produits via API Gateway

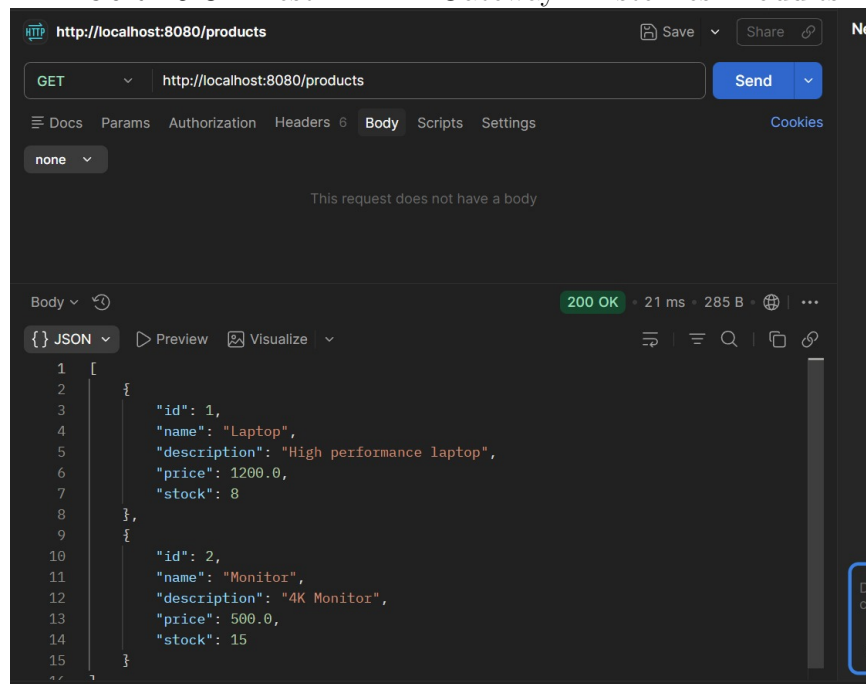
Nous avons testé l'accès aux produits via l'API Gateway.

Requête :

Méthode : GET

URL : `http://localhost:8080/products`

FIGURE 5.3 – Test 2 - API Gateway - Lister les Produits

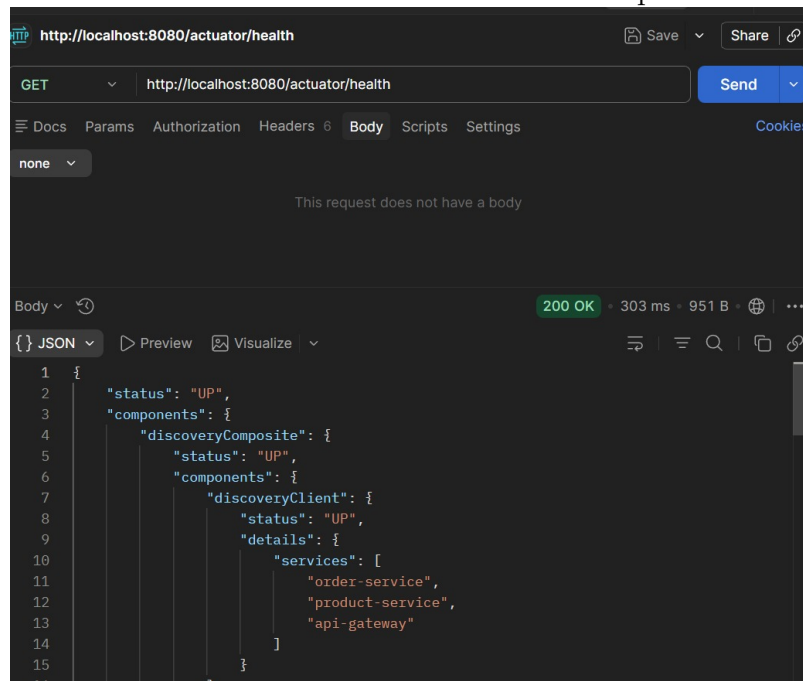


L'API Gateway route correctement la requête vers le Product Service et retourne les produits avec le stock actuel.

6 Tests Finaux

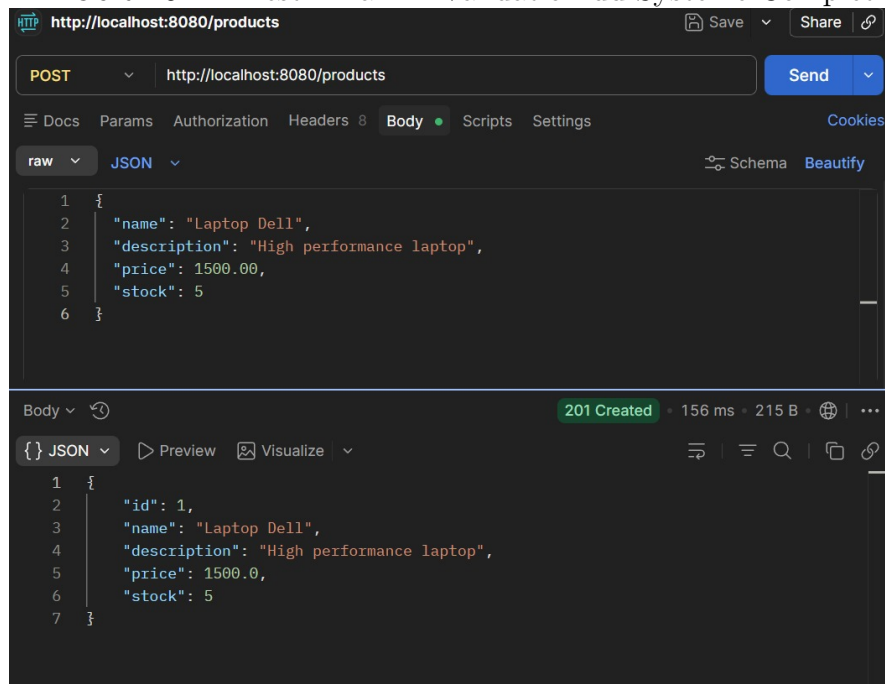
6.1 Test Final 1 : Vérification Complète du Flux

FIGURE 6.1 – Test Final 1 - Vérification Complète du Flux



6.2 Test Final 2 : Validation du Système Complet

FIGURE 6.2 – Test Final 2 - Validation du Système Complet



7 Résultats et Conclusions

7.1 Résumé des Tests Réalisés

Tous les tests ont été exécutés avec succès :

- Dashboard Eureka - État initial
- Product Service - Création de produit
- Product Service - Récupération des produits
- Order Service - Création de commande
- Order Service - Récupération des commandes
- Exercice 1 - Gestion du stock (4 tests)
- Exercice 2 - API Gateway (2 tests)
- Tests finaux (2 tests)

7.2 Fonctionnalités Implémentées

Fonctionnalités principales :

1. Service Discovery avec Eureka
2. CRUD complet pour les produits
3. CRUD complet pour les commandes
4. Communication inter-services (Feign)
5. Gestion du stock avec décrémentation
6. Validation du stock insuffisant
7. API Gateway pour centraliser l'accès
8. Gestion des erreurs

7.3 Technologies Utilisées

Domaine	Technologie	Version
Framework	Spring Boot	3.2.5
Cloud	Spring Cloud	2023.0.0
Discovery	Eureka Server	Netflix
Gateway	Spring Cloud Gateway	4.1.5
Communication	OpenFeign	REST
Database	H2 / JPA	En-mémoire
Build	Maven	3.x
Testing	Postman	Latest

TABLE 7.1 – Technologies utilisées

7.4 Apprentissages Clés

Compétences acquises :

1. Architecture microservices avec Spring Cloud
2. Découverte dynamique de services avec Eureka
3. Communication inter-services via REST et Feign
4. Gestion des données distribuées
5. API Gateway et routage intelligent
6. Gestion de la cohérence des données
7. Tests d'APIs REST avec Postman

Conclusion Générale

Ce projet a démontré avec succès la mise en place d'une architecture microservices complète avec Spring Cloud. Les trois services (Eureka, Product Service, Order Service) communiquent efficacement et gèrent les opérations CRUD de manière distribuée. L'implémentation de la gestion du stock et de l'API Gateway ont ajouté des fonctionnalités avancées répondant aux besoins réels d'une application e-commerce.

Points clés de réussite

- Architecture scalable et découplée
- Communication entre services fluide via REST et Feign
- Gestion cohérente du stock à travers les services
- Validation robuste des entrées utilisateur
- API Gateway centralisant l'accès
- Respect des bonnes pratiques SOA
- Tests complets avec Postman
- Documentation professionnelle complète

Qualité de la mise en œuvre

La réalisation de ce TP a permis de :

- Maîtriser Spring Boot et Spring Cloud
- Développer une compréhension profonde des microservices
- Apprendre à concevoir des systèmes distribués fiables
- Acquérir de l'expérience pratique avec Eureka et Feign
- Maîtriser le testing d'APIs REST
- Implémenter des fonctionnalités métier complexes

Date de soumission : 03 Avril 2026

Étudiante : LAZNASNI RAYHANA

Professeur : Pr. ELBAGHAZAOUI BAHAA

Institution : ENSA BM - Benimellal